Design, Validation and Verification Document

Version 0
Group 15
COMPSCI 4ZP6
Dr. Mehdi Moradi

Name	**Email**	**Role**
Selina Kim	kim391@mcmaster.ca	Full-stack developer, QA
Giovanna Gerada	geradag@mcmaster.ca	Program Manager, UX/UI Designer, Full-stack developer
Marwa Khafagy	khafam1@mcmaster.ca	Backend Developer, QA
Stella Gu	gus46@mcmaster.ca	Backend Developer, Data Engineer
Tinu Joseph	josept12@mcmaster.ca	Full-stack developer, UX/UI Designer

1. Purpose Statement **1**
2. Component Diagram **2**
3. Relationship between Components and Requirements **2**
3.1 Frontend 2
3.2 Backend 2
4. Project Components **3**
4.1 Frontend 3
4.2 Cloudflare Tunnel 5
4.3 Backend Server 5
4.4 Database Storage 9
4.5 External Services 10
5. User Interface **10**
6. Component Test Plan **10**
6.1 Frontend Testing 10
6.2 Cloudflare Tunnel Testing 11
6.3 Backend Server Testing 11
6.4 Database Storage Testing 13
6.5 External Services Testing 13
Appendix **14**
Glossary 14
Component Diagram 14
UI Design 14

1. Purpose Statement

The purpose of this project is to reduce friction in vocabulary acquisition for language learners by developing a mobile application that automatically generates flashcards from user-provided verbal or typed input. Language learners frequently encounter unfamiliar words during everyday activities such as reading or online communication, but the effort required to manually record, define, and organize these words often leads to abandoned study intentions and lost learning opportunities. The proposed system streamlines this process by creating flashcards that include translations and example sentences in context, allowing users to focus on learning and spend more time reviewing the flash cards rather than creating them.

2. Component Diagram

Please see the Appendix for the component diagram.

3. Relationship between Components and Requirements

3.1 Frontend

Component(s)	Requirements Covered
Login View	P0: User Authentication (Google OAuth, login, log out, sign up) P0: Secure user authentication
Home View	P0: Home Page (used to access all other pages) P2: Users can see their stats (e.g., words learned, review/login streak, etc.)
Deck View	P0: Individual Deck View (all cards in one deck) P0: Decks (all decks for all languages) P0: Deck Editing Frame (Add/Remove/Edit Cards) P0: Users will have the ability to create, view, and organize flashcards in a deck. P1: Update all deck frames to incorporate an image for nouns P1: Adding import/export to deck view
Review View	P0: Deck Revision (algorithm used to review cards) P0: Users should be able to review flashcards (word → translation + example)
Profile View	P1: Account Information Page (username, account creation date) P2: View user progress and learning statistics
Notifications View	P2: User gets a pop-up notification if they have not reviewed their cards in x amount of time
Help View	P0: Help Page (include information about how to use the app)

3.2 Backend

Component(s)	Requirements Covered
Authentication, Google OAuth (External Service)	P0: App supports user management (create, delete, modify of user info) P1: App is secure: A user's decks can't be viewed by other users. Login information is protected
Database Integration, PostgreSQL Database, MinIO Database	P0: App will have a database for deck storage and user management P0: Storage is offline and locally cached P0: User should be able to create/delete/edit the generated flashcard P1: Storage is synced with a custom cloud server P1: Users can import/export their decks to expand their deck library P1: App will have select settings fully implemented P2: User Progress Tracking (Store/retrieve learning statistics.)

	P2: Send notifications if users have cards due or haven't revised in a while P2: Users will be able to share their decks via a link for viewing P3: Users can set their decks as public or private (default being private) P3: Allow users to create and edit collaborative decks
FSRS Algorithm, FSRS Optimizer	P0: Users should be able to review their decks and flashcards via a spaced repetition algorithm
Image Generation, External Service Integration	P1: Images should be generated for cards that are nouns P2: If a card is a non-noun, an image representing the word should be generated for the card
Dictionary, Translation, External Service Integration	P0: App will have bilingual translation support (card generation) P0: Given a word, the program will make a flash card with translation and relevant examples P1: App will have multi-language translation support (card generation) P2: User can make a card in language A for recall in language B and choose to reverse which side they recall (auto-generation supports both ways) P3: Extend multi-language support to additional languages
STT, External Service Integration	P2: Application will have STT functionality for card generation (say a word, generate a card)
TTS, External Service Integration	P0: Application will have TTS functionality for card revision

4. Project Components

4.1 Frontend

4.1.1 Profile View

Normal Behaviour: User clicks profile icon in header to view their account information, preferences, and settings; displays user's decks, progress statistics, and allows account management.
API: Check 4.3.7 for API requirements of user logout and fetching user information.
Implementation: Accessed via ProfileIcon button in root layout header. When tapped, should display full profile component with user data display and settings management.
Potential Undesired Behaviours: Token expiration during profile view could log users out unexpectedly, network errors may show stale cached data or fail to load user information.

4.1.2 Home View

Normal Behaviour: Landing page after login showing user dashboard with study statistics, recent decks, upcoming reviews, and quick action buttons to start studying.
API:
GET /home/dashboard: *Fetch user's home dashboard data*

- **Req:** JWT; **Res:** {total_due_cards, past_statistics}

Implementation: Displays dashboard widgets, statistics, and navigation to deck/review flows.
Potential Undesired Behaviours: Dashboard statistics could be slow to load with large datasets; real-time updates for study progress may cause unnecessary re-renders.

4.1.3 Deck View

Normal Behaviour: Displays list of user's flashcard decks with options to create, edit, delete decks. Shows NoDecksBanner when empty. Allows deck selection for studying or management.
API:
GET /decks: *Fetch all decks of a user*

- **Req:** JWT **Res:** {decks: {d_id, deck_name, word_lang, trans_lang, card_count, description}[]}

GET /decks/:d_id: *Fetch specific deck from user*

- **Req:** JWT; **Res:** {d_id, deck_name, word_lang, trans_lang, description, card_count, c_id[]}

POST /decks: *Create a new deck*
- **Req:** JWT, {deck_name, word_lang, trans_lang, description?}; **Res:** {message, deck}

POST /decks/:d_id: *Edit existing deck*
- **Req:** JWT, {deck_name?, description?}; **Res:** {message, deck}

DELETE /decks/:d_id: *Delete existing deck*
- **Req:** JWT; **Res:** {message}

POST /decks/:d_id/card: *Create a card in specific deck*
- **Req:** JWT, {word, translation, definition, word_example, trans_example?, image?, word_audio?, trans_audio?, word_roman, trans_roman?}; **Res:** {message, card}

GET /decks/:d_id/cards/:c_id: *Fetch data of a specific card*
- **Req:** JWT; **Res:** {c_id, d_id, word, translation, definition, word_example, trans_example, image, word_audio, trans_audio, word_roman, trans_roman}

POST /decks/:d_id/cards/:c_id: *Edit existing card*
- **Req:** JWT, {word?, translation?, definition?, word_example?, trans_example?, image?, word_roman?, trans_roman?}; **Res:** {message, card}

DELETE /decks/:d_id/cards/:c_id: *Delete existing card*
- **Req:** JWT; **Res:** {message}

For API requirements of translation and STT functionality, check 4.3.4 and 4.3.5.
Implementation: Shows NoDecksBanner with CreateNewDeckModal trigger when no decks exist. Otherwise displays deck list components, API integration, and deck management.
Potential Undesired Behaviours: Creating multiple decks rapidly could cause race conditions; deleting deck while actively studying it may cause navigation errors or data inconsistencies.

4.1.4 Review View

Normal Behaviour: Presents flashcards for spaced repetition review showing word/phrase, definition, and user rates difficulty to schedule next review using spaced repetition algorithm.
API:
GET /review/decks: *Fetches decks with due cards*
- **Req:** JWT; **Res:** {decks: {d_id, deck_name, word_lang, trans_lang, cards_due}[]}

POST /review/start: *Starts review session*
- **Req:** JWT, {deck_id}; **Res:** {session_id, cards: {c_id, r_id, word, translation, word_example, trans_example, word_audio, trans_audio, word_roman, trans_roman, is_new_card, last_review_date?}[], total_cards, current_card_index}

POST /review/submit: *Submits card rating*
- **Req:** JWT, {session_id, c_id, rating, recall_time} **Res:** {message, updated_review: {r_id, next_due_date, interval, difficulty, stability}, next_card?, current_card_index, cards_remaining, session_complete}

For API requirements of translation and STT functionality, check 4.3.6.
Implementation: Shows flashcard display component, card animations, spaced repetition logic, and tracks progress. Display the Go Home and Keep Studying button when done,
Potential Undesired Behaviours: Network latency during card submission could cause issues if user taps multiple times. Offline usage without sync may lose review progress.

4.1.5 Help View

Normal Behaviour: Displays app documentation, FAQs, tutorial guides.
API: Information for Help View will be stored statically on the frontend, so no APIs are used.
Implementation: Shows content sections and collapsible FAQ components.
Potential Undesired Behaviours: Outdated help content could confuse users if not synced with latest app version.

4.1.6 Notifications

Normal Behaviour: Shows system notifications such like review reminders, study streak milestones, and app updates. Users can tap to navigate to relevant content.
API: N/A
Implementation: Uses notification permission handling, push notification setup (Expo Push Service), badge counts, and notification center UI component. Backend scheduled jobs check for notification triggers (due cards, daily reminders) and sends push notification via Expo Push Service to app. Frontend receives and displays notifications as OS-level alert.
Potential Undesired Behaviours: Notification tap navigation may fail if target content is deleted. Timezone mismatches could show reminders at wrong times.

4.1.7 Login

Normal Behaviour: User authenticates via Google OAuth by clicking sign-in button, gets redirected to Google consent screen, returns with token, and app stores JWT for authenticated requests.
API:
POST /auth/google: *Authenticates via Google OAuth*

- **Req:** {id_token}; **Res:** {message, user: {u_id, email, display_name, timezone}, tokens: {access_token, refresh_token}}

POST /auth/refresh: *Refreshes access token*

- **Req:** Refresh JWT; **Res:** {message, tokens: {access_token, refresh_token}}

Implementation: Root layout shows (auth) stack with hidden header. Frontend auth flow uses Expo AuthSession, then passes on data to backend APIs.
Potential Undesired Behaviours: Token refresh failures could force repeated logins. Google API downtime blocks all authentication. Malformed tokens from client could expose security vulnerabilities if not validated properly.

4.2 Cloudflare Tunnel

Normal Behaviour: Provides secure external access to the home-hosted Flask backend by routing HTTPS requests through an outbound Cloudflare Tunnel without exposing the server's public IP. Acts as a reverse proxy that transparently forwards REST/JSON requests and responses between the frontend and backend.
API: Exposes a public HTTPS request/response interface at a Cloudflare-managed hostname. Forwards HTTP methods, paths, headers (including authentication), and bodies directly to backend-defined API routes and returns the backend's responses unchanged.
Implementation: Implemented using the cloudflared service running on the home server, which establishes an outbound encrypted tunnel to Cloudflare. Hostname and route mappings are configured in Cloudflare to forward traffic to the local Flask service.
Potential Undesired Behaviours: Tunnel outages or misconfiguration can prevent external access to the backend. Additional latency may be introduced, and incorrect routing or header handling could disrupt authentication or unintentionally expose endpoints.

4.3 Backend Server

4.3.1 Database Integration

Normal Behaviour: The Database Integration component provides an interface between backend services and persistent storage systems. It handles structured data interactions with the PostgreSQL database and unstructured media storage interactions with MinIO. For structured data, the component receives requests from backend services such as FSRS Algorithm and Speech-to-text, translates them into SQL queries, and returns the corresponding results. For media data, it manages the storage and retrieval of image and audio files by issuing object storage operations to MinIO. This component ensures that user data, decks, cards, review history, scheduling metadata, and associated media assets are consistently stored and retrievable across sessions.

API: The Database Integration component exposes internal service methods used by other backend components rather than public-facing endpoints.
Typical operations include:

- Create, read, update, and delete user records
- Retrieve and update decks, cards, and review data
- Store and query spaced repetition scheduling metadata
- Upload, retrieve, and delete image and audio files via MinIO
- Associate stored media objects in MinIO with application records

Communication with PostgreSQL occurs via SQL queries, while communication with MinIO occurs through S3-compatible PUT and GET operations.
Implementation: We haven't implemented this yet, but it will be implemented as a storage abstraction layer that encapsulates access to both relational and object-based storage systems. PostgreSQL is used for structured relational data, while MinIO is used for storing unstructured media such as images and audio files. The component manages database connections, executes parameterized SQL queries, and interacts with MinIO using an S3-compatible client. Metadata linking media objects to application entities is stored in PostgreSQL, while the binary content itself is stored in MinIO.
Potential Undesired Behaviour: High request volume may lead to increased latency, impacting backend response times. Temporary database unavailability may cause read/write operations to fail, preventing data persistence or retrieval. MinIO outages/network interruptions may block access to image or audio content. Improper cleanup of media objects may result in orphaned files occupying storage space.

4.3.2 FSRS Algorithm

Normal Behaviour: Enables the reviewing of cards in a deck by scheduling their next due date. The next due date for the card is calculated by computing an optimal interval that keeps the card at a desired retention. The card's stability and difficulty values are also updated, and review details are stored in a log. Over time, consistently remembered cards are shown less often.
API: Contains the following public methods:
get_card_retrievability(self, card, current_datetime = None): Calculates the predicted probability that the Card is correctly recalled at the current_datetime.
review_card(self, card, grade, review_datetime = None, review_duration = None): Updates a card's state after a review. Computes the new stability and difficulty, determines the next interval before the card should be reviewed again, and generates a ReviewLog entry for the review.
reschedule_card(self, card, review_logs): Reschedules the given card with the current scheduler provided that card's review logs. Used to reschedule cards after parameters have been optimized.
There are also "private" helper methods to compute stability, difficulty, intervals, clamping, fuzz, etc. as well as API calls to store/obtain Card and ReviewLog information to/from our database.
Implementation: Grade() and LearningState() are IntEnum classes that name integers for grade and learning state. Card() uses LearningState() and is initialized with the following attributes: card_id, learning_state, step, stability, difficulty, due, last_review. ReviewLog() uses Grade() and is initialized with the following attributes: card_id, grade, review_datetime, review_duration. Scheduler() uses all of the above classes and is initialized with the following attributes: parameters, desired_retention, learning_steps, relearning_steps, maximum_interval, enable_fuzzing. This class allows cards to be reviewed and scheduled. Non-IntEnum classes are implemented with Pydantic, supporting import/export with JSON and Python dictionaries.
Potential Undesired Behaviours: The interpretation of the grades may vary, resulting in unideal scheduling. i.e., if a user repeatedly clicks Easy, the interval could grow too fast, resulting in lost cards. Misclicks or non-truthful reviews will also negatively affect accurate scheduling.

4.3.3 FSRS Optimizer

Normal Behaviour: Updates the Scheduler's default parameters by fitting them to the user's review log data, so that future review intervals can be calculated to better match a user's specific memory retention patterns.

API: Contains the following public methods:
compute_optimal_parameters(self, verbose = False): Computes a set of optimized parameters for the FSRS scheduler and returns it as a list of floats.
compute_optimal_retention(self, parameters): Simulates different retention targets and selects the target that minimizes the review cost.
There are also "private" helper methods to compute batch loss, probabilities, costs, etc. as well as API calls to store/obtain Card and ReviewLog information to/from our database.
Implementation: The main class that is of importance for the optimizer is the Optimizer() class. The class is initialized with a user's review_logs and contains methods that use torch.optim.Adam with BCELoss() across batches of the review_log entries to update the model's parameters. The Optimizer() uses classes of the FSRS algorithm including Card(), Grade(), ReviewLog(), and Scheduler(). It also uses some of the same constants used by the Scheduler, namely the default parameters and the parameter lower and upper bounds.
Potential Undesired Behaviours: It is possible that if enough review data is not provided, and/or the user is not reviewing cards truthfully and/or timely, the model will be unable to accurately predict the optimal weights, resulting in a potentially unideal setup for the user.

4.3.4 External Service Integration

Normal Behaviour: The External Service Integration component is responsible for handling incoming requests from the Deck View component, sending requests to and handling the response from the External Services components, and sending the result back to the Deck View and Database Integration components.
API:
GET /define/<word>
- **IN:** English word in URL, API key
- **OUT:** Dictionary containing: English definition, pronunciation, audio URL, definition(s), and example sentences

POST /translate
- **IN:** Required: text, target language, API Key; Optional: source language
- OUT: Dictionary containing: detected source language, text in the target language

GET /image
- **IN:** Photo query, API Key
- **OUT:** Dictionary Containing: Photo URL

Implementation: This component has a controller and service layer that handle communication between our frontend and 3 external APIs. The controller defines routes and handles HTTP input and output, while the service layer manages all external API interactions. For dictionary integration, the function define_sentence(word) processes requests, returns a JSON response, and calls call_dictionary_api(word) which extracts relevant information, formats the response, and handles fallback logic from the external dictionary API. For translation integration, translate() processes requests, validates required parameters, and returns JSON responses. The translation logic is managed by a DeepL client which calls translate_text(text, [source_lang], target_lang) to translate the text. For image integration, the function get_image() processes requests and validates data before returning an appropriate JSON response. It calls call_image_api(query) which retrieves the image URL, and formats the response from the external image API. For all 3 integrations, a method such as save_translation() passes the result to the Database Integration component before the HTTP response is returned to the client.
Potential Undesired Behaviours: Network failures or timeouts may lead to slow responses for HTTP errors. API quota exhaustion can result in denied requests. Unexpected API responses can cause parsing errors or for the component to break. Performance bottleneck if there are too many requests made at the same time. Partial failures such as when the client response succeeds but saving to the database fails.

4.3.5 Speech-to-Text

Normal Behaviour: The STT component is responsible for handling incoming requests from the Deck View component, transcribing audio to text, and sending that response back to the Deck View and Database Integration components.

API:

POST /stt	**IN:** Audio File, language	**OUT:** List containing transcribed text

Implementation: This component has a controller and service layer that handle communication between our frontend and STT. The controller layer defines the route for transcribing audio, handling HTTP input and returning text or JSON metadata. It also calls save_text() to pass the generated text to the database integration component before returning the HTTP response to the client.

Potential Undesired Behaviours: Processing failure due to receiving missing, invalid, oversized, or unsupported audio files. Poor transcription caused by incorrect language input or low-quality audio. Partial failures such as when transcription to the client succeeds but saving to the database fails.

4.3.6 Text-to-Speech

Normal Behaviour: The TTS component is responsible for handling incoming requests from the frontend components, sending the result back to the frontend, and sending the result to the database integration component.

API:

POST /tts	**IN:** {text, language, speaker_name?, model?, audio_file?}	**OUT:** {definitions, pronunciation, audio_url, examples}
GET /tts/speakers	**IN:** Optional: model name	**OUT:** List of available speakers
GET /tts/languages	**IN:** N/A	**OUT:** List of supported languages
GET /tts/model	**IN:** N/A	**OUT:** List of TTS models

Implementation: This component has a controller and service layer that handle communication between our frontend and the TTS library. The controller layer defines routes for generating speech, listing available speakers, supported languages, and available models, handling HTTP input and returning WAV audio or JSON metadata. It also calls save_audio() to pass the generated audio to the database integration component before returning the HTTP response to the client. The TTSService class in the service layer manages all interactions with the TTS library which includes _get_model() for loading models, generate_speech() for generating speech as NumPy arrays, generate_speech_to_file() for saving audio to files, list_available_models(), and get_speakers().

Potential Undesired Behaviours: Invalid or missing input, audio generation failures, high latency for long text, memory or GPU exhaustion, and unexpected responses or edge cases.

4.3.7 Authentication

Normal Behaviour: The Authentication component acts as a boundary between external identity providers and internal application logic. It receives ID tokens from the frontend, verifies them using Google OAuth services, and maps verified identities to internal user records. The component generates and returns application-specific JWTs that enable stateless authentication for future requests. It also validates JWTs on protected endpoints and supports token refreshing.

API:

Endpoint	**Input**	**Output**
POST /auth/google	Google ID token	User data + access and refresh JWTs
POST /auth/refresh	Refresh JWT	New access and refresh tokens
GET /auth/me	Access JWT	Authenticated user
POST /auth/logout	Access JWT	Logout confirmation

These use REST over HTTPS and JWT-based Authorization headers protocols.

Implementation: The component is implemented using Flask and the Flask-JWT-Extended library. Google ID tokens are validated using Google's OAuth verification libraries. User records are retrieved or created via database integration with PostgreSQL. JWTs are generated locally using backend-managed signing keys, embedding the internal user identifier. Token validation is performed locally on every protected request without external calls.
Potential Undesired Behaviour: Network interruptions may prevent successful authentication requests. External OAuth service downtime will block user login. High authentication request volume may affect backend performance, increasing response times for login and token refresh operations.

4.4 Database Storage

4.4.1 PostgreSQL Database

Normal Behavior: Stores all structured application data (users, decks, cards, review history, scheduling parameters). Makes sure integrity between application objects:

- Each **Decks.u_id** references a valid **Users.u_id**
- Each **Cards.d_id** references a valid **Decks.d_id**
- Each **Review.c_id** references a valid **Cards.c_id** and is unique per card

Stores the MinIO object keys in the **Cards** table, (**image, word_audio, trans_audio**) to allow the backend to fetch media from MinIO, the bucket name shall be stored in the backend and be hardcoded. Fetch all decks for a user, fetch all cards in a deck, fetch review state for a card, and compute due cards based on Review fields.
API:
Inputs: SQL SELECT/INSERT/UPDATE/DELETE statements queried by backend components via Flask.
Outputs: Result sets (rows), generated IDs (serial), row counts, and constraint/validation errors.
Key data structures:

- User records: {u_id, email, display_name, timezone, new_cards_per_day, desired_retention}
- Deck records: {d_id, u_id, deck_name, word_lang, trans_lang, creation_date, last_reviewed, description, is_public, link}
- Card records: {c_id, d_id, word, translation, definition, image, word_example, trans_example, word_audio, trans_audio, word_roman, trans_roman, learning_state, step, difficulty, stability, due_date, last_review, successful_reps, fail_count}
- Review log records: {rl_id, c_id, review_date, review_duration, rating}

Implementation: a PostgreSQL docker container is deployed in docker-compose, on our private server, which can only be accessed by the backend service through our internal docker network.

4.4.2 MinIO Database

Normal Behaviour: MinIO stores and serves all media assets associated with cards such as images and audio files. The backend uploads media to MinIO and later retrieves it using backend generated object keys, which are stored in PostgreSQL, the bucket name is fixed and configured in the backend.
API: Inputs to MinIO are S3 object operations initiated by the Flask backend, primarily PUT (upload), GET (download), HEAD/STAT (existence + metadata), and DELETE (remove). Outputs include object byte streams, metadata such as size, and standard S3-style errors, the key data structure exchanged is the object key string stored in PostgreSQL fields the Cards.image, Cards.word_audio, Cards.trans_audio) mapped to raw media bytes.
Implementation: MinIO is deployed as a Docker container within the project's Docker Compose stack with persistent volume storage for objects. The Flask backend connects to MinIO using an S3 client configured via endpoint and credentials. A hardcoded bucket name will be in the backend. Object keys are generated by the backend and kept in PostgreSQL.
Potential Undesired Behaviours: Media references can break if PostgreSQL stores an object key that does not exist in MinIO, resulting in missing images/audio at runtime. Misconfigured credentials, bucket name, or network connectivity can prevent uploads/downloads, and incorrect Content-Type

metadata can cause clients to display or play media incorrectly. If deletes are not handled consistently, MinIO may accumulate orphaned objects even after the related database records are removed.

4.5 External Services

4.5.1 Image Generation, Dictionary, Translation

Component details for image generation, dictionary, and translation have been covered in the External Services Integration component.

4.5.2 Google OAuth

Normal Behaviour: The Google OAuth component provides third-party identity verification. It authenticates users using Google credentials and issues signed ID tokens that are linked to the user's identity. These tokens are later verified by the backend *Authentication* component to establish trust. Google OAuth does not interact directly with internal application state or permissions.
API: Uses Google OAuth Authorization, Google Token Verification and Public Key Endpoints. Token format is OpenID Connect ID Token (JWT), signed by Google and verified using public keys.
Implementation: Google OAuth is implemented as an external identity provider using the OAuth 2.0 and OpenID Connect standards. The frontend interacts with Google to retrieve an ID token, while the backend uses Google's verification APIs and public certificates to validate token integrity, issuer, audience, and expiration.
Potential Undesired Behaviours: Token verification failure due to expired or revoked tokens, network unavailability of Google OAuth services, misconfigured OAuth client IDs, user denial of consent, overall dependency on third-party availability and policy changes

5. User Interface

Images of the Figma that outline the UI design for each page of our application are in the Appendix. The title font and primary colours were chosen with a pasta theme in mind. We also avoid pure blacks and whites to ease eye strain. The font choices keep readability in mind across different languages. The placement of items attempts to mimic popular design choices in other applications, such as a profile icon in the header, page icons in the footer, intuitive icons, grayed out text fields, outlined boxes, shadows, etc. to cleverly separate a page's components and encourage user interaction through a clean design that feels intuitive.

6. Component Test Plan

6.1 Frontend Testing

Unit tests: Use Jest and the React Native Testing Library for frontend testing.

- Components: Test all React Native components in isolation to verify rendering behavior. Mock API calls and navigation to ensure components handle different data states (loading, success, error, empty). Verify user interactions (button clicks, form submissions, swipe gestures) trigger expected callbacks. Test conditional rendering logic (e.g., "No Decks" banner vs deck list)
- Custom Hooks: Test authentication hooks (token management, session persistence). Test data fetching hooks (API integration, cache invalidation).
- Utility Functions: Test form validation logic and data transformation functions (e.g., formatting timestamps based on user timezone).
- Special Considerations: Google OAuth flow integration. Card flip animations and review rating submission. Deck/card CRUD operations with optimistic UI updates. Navigation between tabs and screens.

Performance tests and metrics: Use React Native Performance Monitor, Expo performance profiling.
API Response Latency:

- API response time for critical paths (dashboard load, review session start, card submission, fetch deck/card data, etc)
    - Target: < 500ms for dashboard load, < 300ms for card submission during review

UI Render Performance:

- Frame rate during animations and scroll
    - Target: Maintain 60 FPS during card flips, transitions, and deck list scrolling
- Time to Interactive (TTI) for each screen
    - Target: < 1 second for Home/Decks views, < 1.5 seconds for Review session start

Memory Usage:

- App memory footprint during typical usage (prevent crashes and ensure app stability)
    - Target: < 150MB during normal operation, no memory leaks

App Cold Start Time:

- Time from app icon tap to first interactive screen
    - Target: < 3 seconds on mid-range devices

Network Efficiency:

- Number of API calls per user session and data transferred (reduce data usage for users on limited mobile plans and improve offline capability)
    - Target: Cache dashboard data for 5 minutes, minimize redundant card fetches

6.2 Cloudflare Tunnel Testing

Unit tests: Verify that backend requests are correctly routed through the Cloudflare Tunnel and reach the Flask API with headers and payloads preserved. Tests ensure authentication headers are forwarded correctly, expected routes are accessible via the public hostname, and invalid paths or methods return appropriate HTTP errors. Additional tests cover tunnel-unavailable scenarios to confirm the backend fails safely without exposing internal network details.
Performance tests and metrics: Validate E2E request/response behaviour using real HTTPS calls through the Cloudflare Tunnel. Metrics include HTTP status codes, average and p95 response latency, and request success rates for common API endpoints. These tests ensure the tunnel introduces acceptable overhead while providing reliable external access to the home-hosted backend.

6.3 Backend Server Testing

6.3.1 Database Integration

Unit tests: Unit tests for the Database Integration component are implemented using the Pytest framework and focus on verifying correct data persistence and retrieval logic. For PostgreSQL, unit tests validate the correct behaviour of CRUD operations across the Users, Decks, Cards, and ReviewHistory tables. These tests verify that records are correctly created, retrieved, updated, and deleted, and that foreign key relationships and cascading deletes behave as expected (e.g., deleting a deck removes its associated cards and review history). Additional test cases ensure that database constraints are enforced, such as unique deck names per user and non-null fields.
For MinIO integration, unit tests mock object storage operations to verify that image and audio files are uploaded, retrieved, and deleted correctly, and that returned object paths are properly associated with card records in PostgreSQL. Tests also cover failure scenarios such as missing media files, invalid paths, or partial failures where database updates succeed but media operations fail.
Performance tests and metrics: Performance testing for the Database Integration component focuses on query latency, throughput, and storage operation responsiveness. For PostgreSQL, performance tests measure the execution time of common queries such as retrieving all decks for a user, fetching cards for a deck, querying due cards for review, and inserting review history records during active study sessions. The system aims to keep average query latency under 100 ms for typical read operations and under 200 ms for write operations involving reviews and scheduling updates, even when handling users with large decks. For MinIO, performance tests measure upload and retrieval times for image and audio files of varying sizes. Target metrics include maintaining upload and download latencies under 500 ms for small media files commonly used in flashcards. E2E integration tests combine PostgreSQL and MinIO operations to evaluate scenarios where card creation involves

both database writes and media uploads. These tests help identify performance bottlenecks and ensure that combined storage operations remain responsive during normal usage.

6.3.2 FSRS Algorithm

Unit tests: Pytest unit tests focus on ensuring cards are reviewed and rescheduled correctly. These test cases will include scenarios like: ensuring the intervals of a card, stability, difficulty, and retrievability are correctly calculated over a series of reviews, that stability and difficulty values stay within their min/max bounds, that timestamps are correctly handled in UTC, and that classes support import/export with JSON and Python dictionaries without loss of data.
Performance tests and metrics: We expect the FSRS algorithm to process the review of a single card in under 10ms. We expect 10,000 cards to be rescheduled in under 3s. We also expect the importing/exporting of cards (JSON or Python dictionary) to be under 10ms.

6.3.3 FSRS Optimizer

Unit tests: Pytest unit tests focus on ensuring new parameters are calculated correctly. Test cases will include scenarios like: ensuring parameters are optimized for sufficient review logs, that optimized parameters correspond to the desired retention, that interval simulation with different desired retentions behave as expected, that the optimized parameters are reproducible, different from default parameters, have a better batch loss than default parameters, etc.
Performance tests and metrics: The predictions our model makes depend on review histories. As a result, the performance is variable between users, depending on the number and quality of reviews. Still, we aim to keep runtimes under 3 seconds for reviews with under 10k entries. We also plan to measure the performance of our optimizer by splitting a test review log into a 80/10/10 training, validation, test split in order to evaluate performance and confirm that the model is not being overfit. We will use BCELoss as our loss function, and use AUC-ROC and ECE as primary evaluation metrics.

6.3.4 External Service Integration

Unit tests: The focus is on verifying the application logic without making real API calls and are built using the Pytest framework. For the dictionary service, unit tests mock responses from the dictionary API to confirm correct parsing of definitions, pronunciations, example sentences, and audio URLs. They also test handling of partial or missing data, network failures, multiple definitions, and word-not-found scenarios with suggestions. Translation service unit tests mock the translation API to verify correct handling of auto-detected and explicit source languages, Unicode characters, missing parameters, and empty input strings. Similarly, image service unit tests mock API responses to ensure the correct processing of valid image queries and handling of empty results. In addition, all external services will have tests to handle network failures, including unreachable services, request timeouts, and API quota exhaustion scenarios.
Performance tests and metrics: For external services, latency is measured for data retrieval and we expect responses to take <1s. Integration tests using real API calls are used to provide realistic data for these metrics. Throughput tests evaluate system behaviour under increasing request volumes to identify performance bottlenecks when handling concurrent requests. Partial failure cases are tested to ensure the client response is not blocked and that persistence failures are properly logged or queued for retry.

6.3.5 Speech-to-Text

Unit tests: At the controller level, tests validate request handling, including missing or invalid audio files, unsupported file formats, oversized payloads, and invalid language parameters, ensuring appropriate HTTP error codes are returned. Service-level unit tests focus on correct interaction with the STT engine, including successful transcription, handling of empty or low-quality audio, timeout scenarios, and malformed responses from the external STT provider. Additional tests verify that database persistence is invoked correctly, that failures in save_text() do not block the HTTP response, and that duplicate or repeated requests are handled appropriately.
Performance tests: Latency is measured E2E from request receipt to transcription response, with targets defined for average and worst-case response times under normal and peak loads. Throughput

tests evaluate the number of concurrent transcription requests the system can handle without degradation. For transcription quality, appropriate STT metrics such as Word Error Rate (WER) or Character Error Rate (CER) are used as they better reflect transcription performance, especially for longer or noisy audio inputs. Partial failure cases are tested to ensure the client response is not blocked and that persistence failures are properly logged or queued for retry.

6.3.6 Text-to-Speech

Unit tests: Verify the TTS logic under various conditions, including successful audio generation with specified or default speakers, Unicode text handling (Korean, Japanese, special characters, emojis), and proper error handling for missing parameters, unsupported languages, empty text, empty audio, or service exceptions. Additional tests cover retrieval of speakers, models, and supported languages, as well as default speaker fallbacks.
Performance tests and metrics: Validate E2E with real TTS model calls, check audio generation for multiple languages, speaker and model selection, and proper WAV formatting (sample rate, length, data type). Measure response times for short text inputs which we expect to take under 10s. Metrics such as HTTP status codes, audio file properties, and request duration are used to ensure both functional correctness and performance reliability. Partial failure cases are tested to ensure the client response is not blocked and that persistence failures are properly logged or queued for retry.

6.3.7 Authentication

Unit Tests: Verify the user authentication logic under various conditions, including successful Google OAuth token verification with valid credentials, JWT token generation and refresh operations, and proper error handling for missing tokens, invalid tokens, expired tokens, malformed authorization headers, and authentication service exceptions. These tests ensure correctness of API behaviour without relying on real Google OAuth services.
Performance tests and metrics: Validate E2E functionality with real Google OAuth tokens, checking the complete authentication flow for login, token refresh, and logout operations. They verify JWT token structure validation (proper header.payload.signature format with two dots), successful user profile retrieval with valid access tokens, token refresh generating new valid access tokens different from the original, and logout successfully processing with valid tokens. Metrics such as HTTP status codes, JWT token format validation are used to ensure both functional correctness and reliability.

6.4 Database Storage Testing

Unit tests: Verify database interaction logic for all core entities (Users, Decks, Cards, Review) under various conditions, including successful creation, retrieval, update, and deletion of records through the backend data-access layer. Tests validate enforcement of schema constraints such as primary keys, foreign keys, unique constraints (e.g., unique deck names per user) and valid email formats. Additional tests ensure cascade delete behaviour functions correctly (e.g., deleting a user removes associated decks, cards, and reviews), and that review records maintain 1-to-1 relationships with cards. Tests also verify correct storage and retrieval of MinIO object keys for images and audio fields without relying on the presence of actual MinIO objects.
Performance tests and metrics: Validate E2E database functionality using a real PostgreSQL instance, measuring correctness and responsiveness of common query patterns such as fetching user decks, retrieving cards within a deck, and selecting due review cards based on scheduling fields. Performance metrics include query execution time, transaction latency for write operations (deck/card creation and review updates), and HTTP response times for database-backend API endpoints. These tests ensure that database operations complete within acceptable time bounds and that relational queries scale reliably as the number of users, decks, cards, and reviews increases.

6.5 External Services Testing

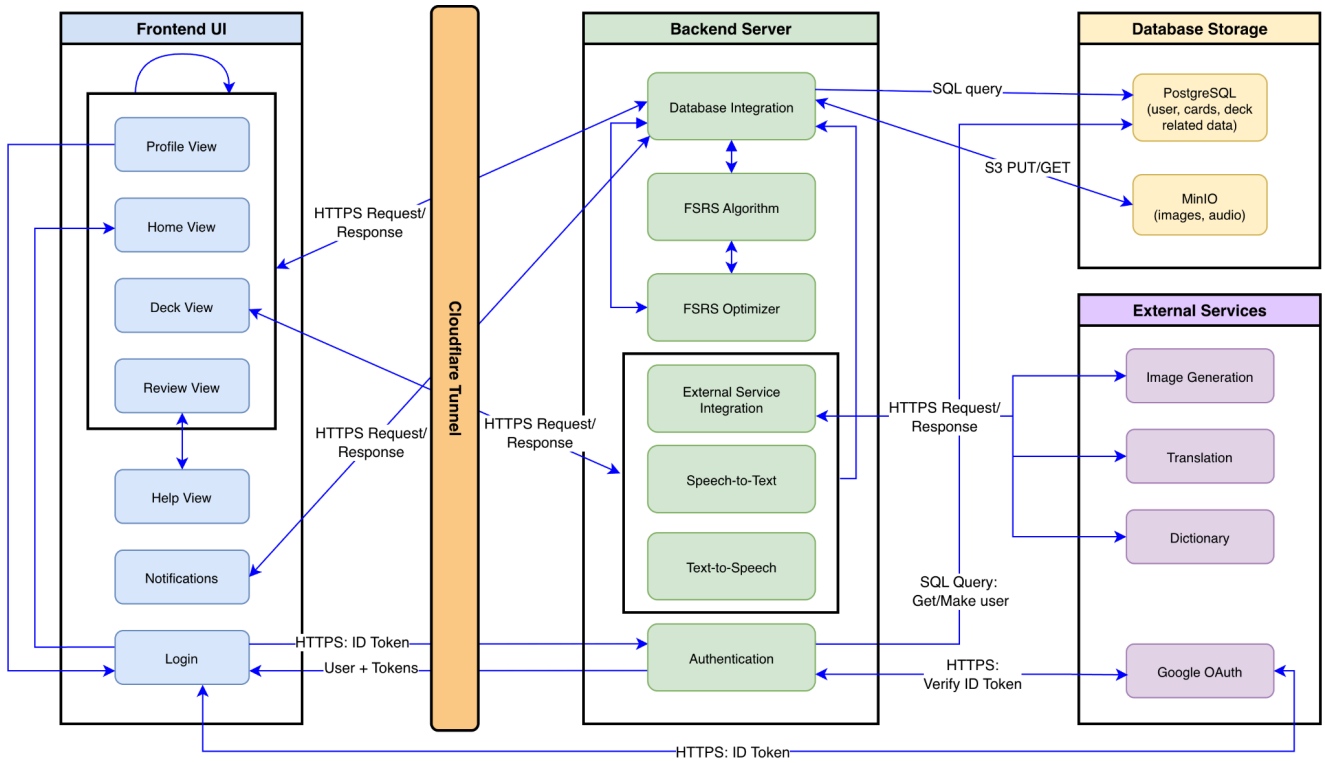
Tests for components in External Services have been covered in the External Services Integration component and the Authentication component.

Appendix

Glossary

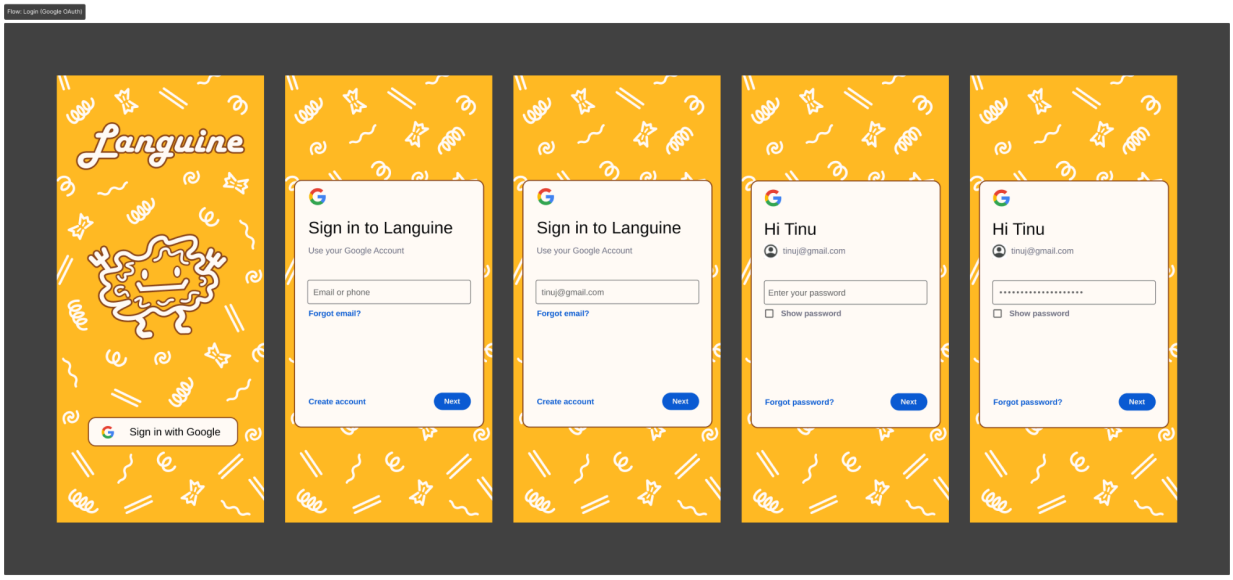
E2E	End-to-end
STT	Speech-to-Text
TTS	Text-to-Speech
ECE	Expected Calibration Error
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BCELoss	Binary Cross Entropy Loss

Component Diagram

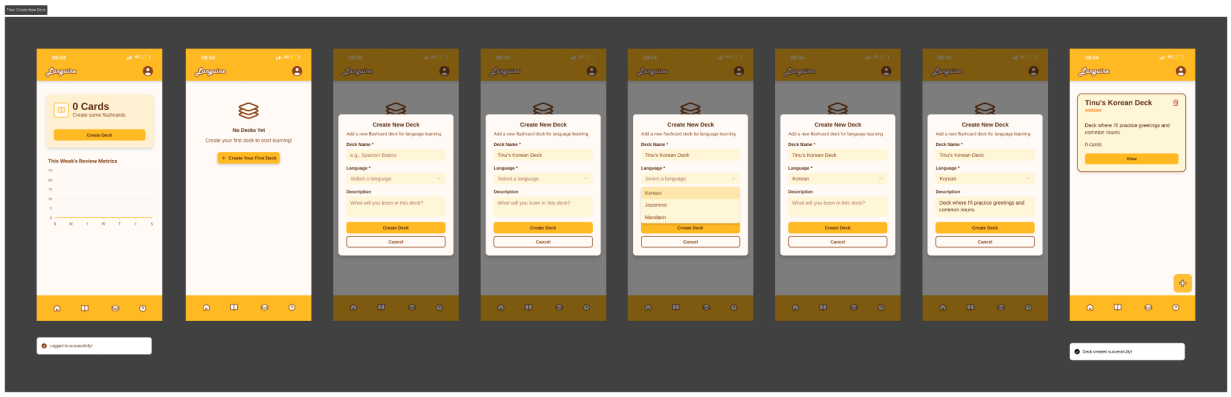


UI Design

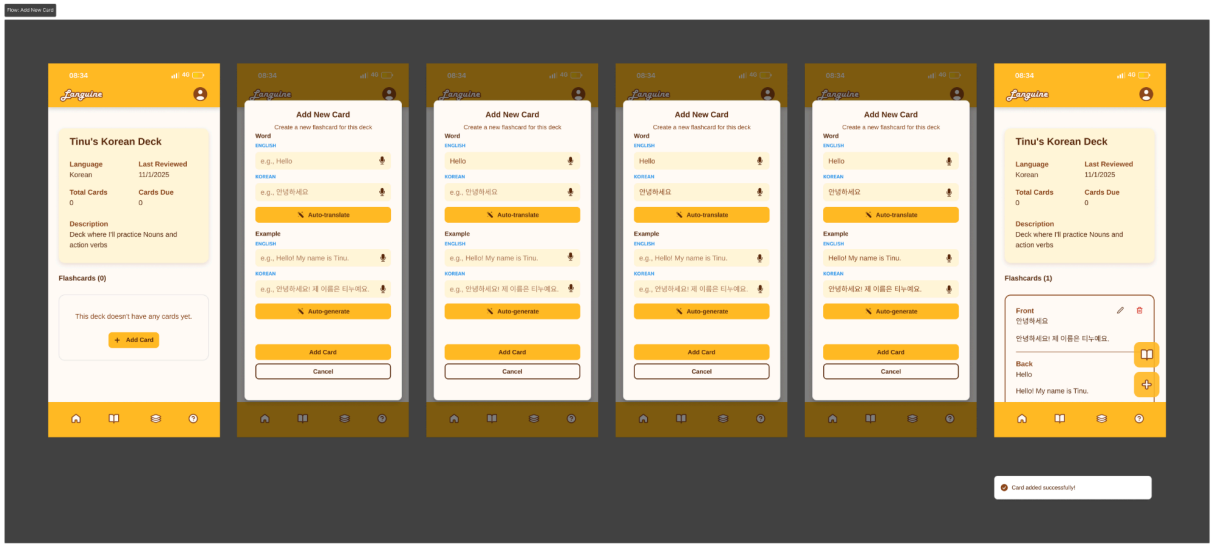
We also link access to the Figma so you can see the designs/workflows in more detail here.



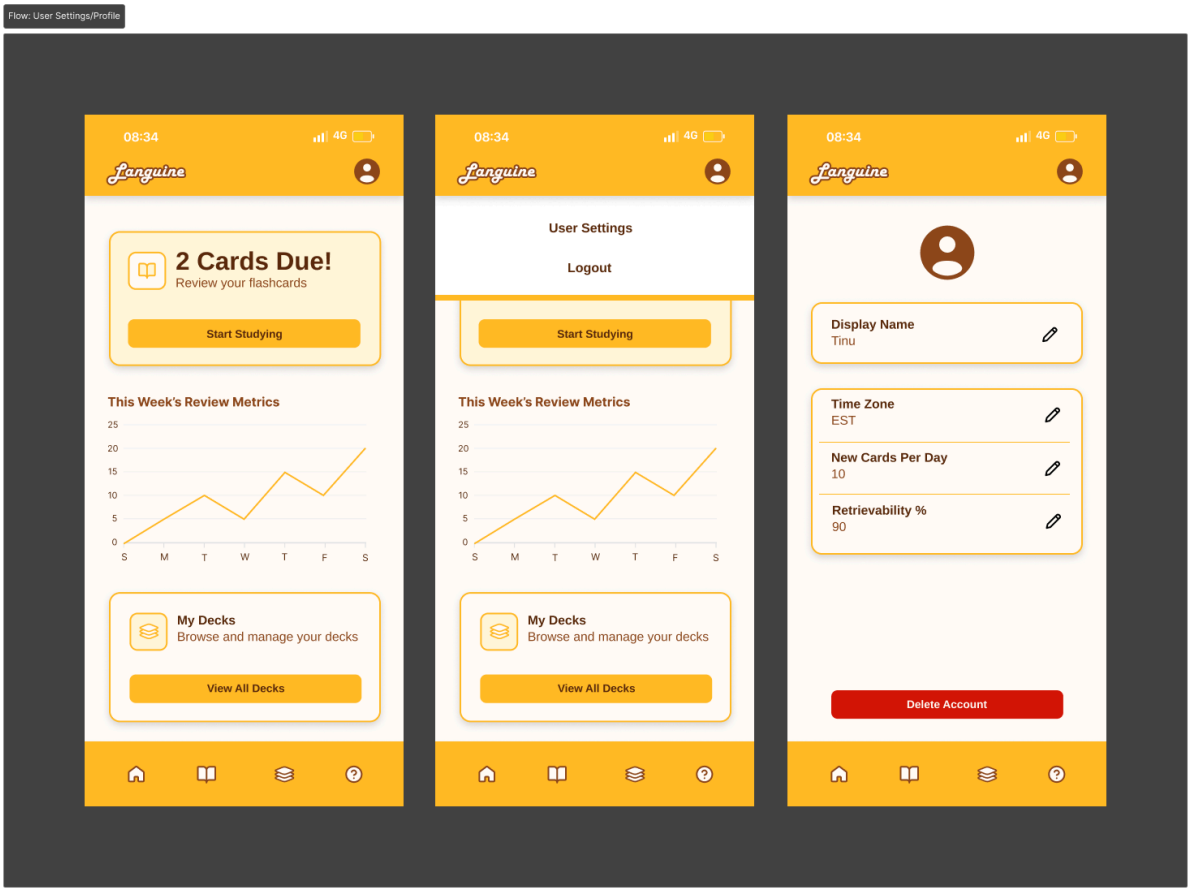
Login Page with login workflow



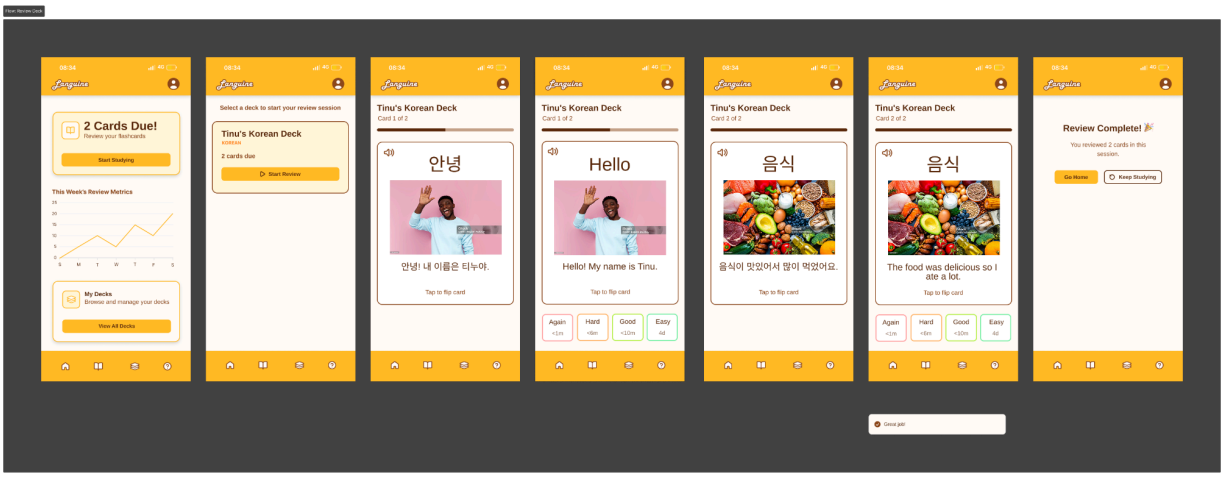
Home and Deck Page with create new deck workflow



Specific Deck View with add new card workflow



Home Page with metrics and User Settings/Profile Page



Review Deck Page with deck review workflow

Home Page post review and Help Page